Hands-on Exercise 1 : Token Economics

GPT-5.x & O1/3 | **GPT-4 & GPT-3.5 (legacy)** | GPT-3 (legacy)

Frontend Assessment - 1
Real-Time Stock Market Updates
In this assessment, your objective is to create a real-time stock market application using HTML/SCSS frontend and integrate it with JS for live updates.
Requirements
- Viewport Layout:
  - The viewport should consist of two tabs with width ratios of 3:1
  - Tab1 (Main) = 75% and Tab2 (Side) = 25%
  - Main will have a bar-graph container
- Height of bar-graph container: 500px

Clear | Show example

Tokens 586 **Characters** 2215

```
Frontend Assessment - 1
Real-Time Stock Market Updates
In this assessment, your objective is to create a real-time stock market
 application using
HTML/SCSS frontend and integrate it with JS for live updates.
Requirements
• Viewport Layout:
◦◦ The viewport should consist of two tabs with width ratios of 3:1
▪ Tab1 (Main) = 75% and Tab2 (Side) = 25%
▪ Main will have a bar-graph container
• Height of bar-graph container: 500px
• Max number of bars : (Main-width/20px)
```

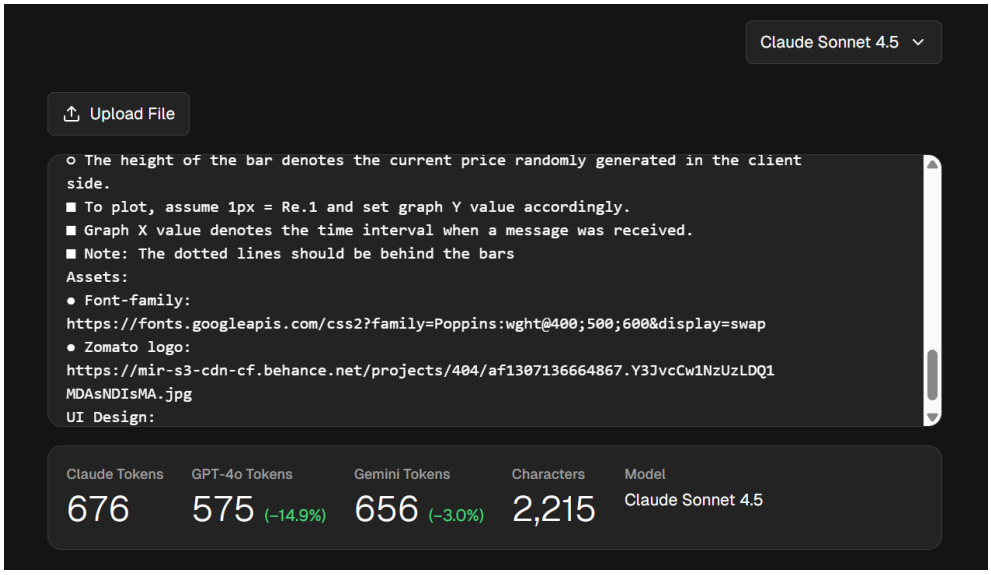
GPT-5.x & O1/3 | GPT-4 & GPT-3.5 (legacy) | **GPT-3 (legacy)**

Frontend Assessment - 1
Real-Time Stock Market Updates
In this assessment, your objective is to create a real-time stock market application using HTML/SCSS frontend and integrate it with JS for live updates.
Requirements
- Viewport Layout:
  - The viewport should consist of two tabs with width ratios of 3:1
  - Tab1 (Main) = 75% and Tab2 (Side) = 25%
  - Main will have a bar-graph container
- Height of bar-graph container: 500px

Clear | Show example

Tokens 640 **Characters** 2215

```
Frontend Assessment - 1
Real-Time Stock Market Updates
In this assessment, your objective is to create a real-time stock market
 application using
HTML/SCSS frontend and integrate it with JS for live updates.
Requirements
• Viewport Layout:
◦◦ The viewport should consist of two tabs with width ratios of 3:1
▪ Tab1 (Main) = 75% and Tab2 (Side) = 25%
▪ Main will have a bar-graph container
• Height of bar-graph container: 500px
• Max number of bars : (Main-width/20px)
```



<u>Estimated cost for same input:</u>

- For GPT 4 :- $0.001172
- For GPT 3 :- $0.00096
- For Claude Sonnet 4.5: $0.002028

- Use a cheaper model for simple, repetitive and high-volume tasks where cost-saving matters and use an expensive model for complex reasoning, high accuracy and performance.

Hands-on Exercise 2: Model Capability Matrix

<u>Result of text classification :</u>

Model Comparison Table

Feature	BERT	DistilBERT
Model Type	Full transformer (bidirectional encoder)	Distilled (compressed) BERT
Size	Larger	Smaller
Speed	Slower	Faster
Output	Fine-grained (1–5 stars)	Binary (positive/negative)
Confidence Behavior	More balanced	Highly confident predictions
Resource Usage	Higher	Lower

Observations

1. **Prediction Granularity**
    - BERT provides detailed sentiment (e.g., 3 stars for neutral input).
    - DistilBERT provides only binary output, often forcing neutral text into negative or positive.
2. **Handling Neutral Inputs**
    - BERT correctly identifies neutral sentiment (e.g., “The food was okay” → 3 stars).
    - DistilBERT tends to classify neutral inputs as negative with high confidence.
3. **Confidence Levels**
    - DistilBERT shows very high confidence scores, even for ambiguous inputs.
    - BERT distributes probabilities more realistically, reflecting uncertainty.
4. **Performance on Clear Sentiment**
    - Both models perform well on clearly positive or negative sentences.

Conclusions

- **BERT** is more suitable for tasks requiring detailed sentiment analysis and nuanced understanding, such as review scoring or feedback analysis.
- **DistilBERT** is better suited for fast, resource-efficient applications where only basic sentiment classification is needed.

Overall, BERT provides better interpretability and accuracy for nuanced inputs, while DistilBERT offers improved efficiency at the cost of reduced detail.

Result of text generation :

Model Comparison Table

Feature	TinyLlama	GPT-2
Model Type	Lightweight LLM (modern, efficient)	Early-generation transformer
Size	Smaller	Larger than TinyLlama (but older architecture)
Coherence	More structured and descriptive	Less coherent, abrupt transitions
Creativity	Strong descriptive world-building	More narrative but inconsistent
Fluency	Smooth and natural	Occasionally repetitive or disjointed
Output Quality	More controlled and readable	Can become erratic or illogical

Observations

1. **Coherence and Flow**
    - TinyLlama maintains a clear and logical progression (introduction → discovery → new world).
    - GPT-2 shows abrupt shifts in narrative (father, aliens, suicide), reducing coherence.
2. **Story Development**

    - TinyLlama builds a setting with descriptive elements (futuristic city, artifact, new world).
    - GPT-2 introduces multiple plot elements quickly but lacks consistency and structure.
3. **Language Quality**
    - TinyLlama produces smoother and more readable text.
    - GPT-2 contains awkward phrasing and repetition (e.g., repeated references to aliens and actions).
4. **Completion Behavior**
    - TinyLlama stops mid-description but maintains logical continuity.
    - GPT-2 continues longer but drifts into less meaningful or disconnected content.

Conclusions

- **TinyLlama** is better suited for generating structured, coherent, and descriptive text. It performs well in storytelling tasks where clarity and consistency are important.
- **GPT-2** can generate longer narratives but tends to lose coherence and introduce inconsistent or illogical elements.

Overall, TinyLlama provides higher-quality and more reliable text generation, while GPT-2 demonstrates limitations typical of earlier language models.

Hands-on Exercise 3: One Day with AI Challenge

Prompt:

- Generate a Python function to validate email addresses using regex.

Result:

Here's a clean and practical Python function to validate email addresses using a regex pattern:

```python
import re

def is_valid_email(email: str) -> bool:
    """
    Validates an email address using regex.

    Args:
        email (str): Email address to validate

    Returns:
        bool: True if valid, False otherwise
    """
    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    return re.match(pattern, email) is not None
```

How it works:

- `^[a-zA-Z0-9._%+-]+` → username part (before @)
- `@` → mandatory separator
- `[a-zA-Z0-9.-]+` → domain name
- `\.` → dot before TLD
- `[a-zA-Z]{2,}$` → top-level domain (like `.com`, `.org`)

Example usage:

```python
print(is_valid_email("test@example.com"))   # True
print(is_valid_email("invalid-email@"))     # False
print(is_valid_email("user.name@domain.co"))# True
```

Prompt:

- You are an experienced Python developer.
- Write a clean, well-documented Python function that validates whether a given string is a valid email address using regular expressions.
- Requirements:
    - Use Python's built-in re module.

    - The function should be named is_valid_email.
    - It should take a single string argument and return True if the email is valid, otherwise False.
    - Follow common email validation rules (e.g., proper local part, @ symbol, valid domain format).
    - Include inline comments explaining the regex pattern.
    - Provide a few example test cases demonstrating both valid and invalid emails.
- Ensure the code is readable, Pythonic, and follows best practices.

Result:

Here's a clean, well-documented, and Pythonic implementation that follows your requirements:

```python
import re

def is_valid_email(email: str) -> bool:
    """
    Validate whether the given string is a valid email address.
    Args:
        email (str): The email address to validate.
    Returns:
        bool: True if the email is valid, False otherwise.
    """
    # Regex breakdown:
    # ^                          -> Start of string
    # [a-zA-Z0-9._%+-]+          -> Local part: allows letters, digits, and common special characters
    # @                          -> Must contain a single @ symbol
    # [a-zA-Z0-9.-]+             -> Domain name: letters, digits, dots, and hyphens
    # \.                         -> Literal dot before the top-level domain (TLD)
    # [a-zA-Z]{2,}               -> TLD: at least 2 alphabetic characters (e.g., .com, .org)
    # $                          -> End of string
    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
```

```python
    # Use fullmatch to ensure the entire string matches the
pattern
    return re.fullmatch(pattern, email) is not None
# ----------------------------
# Example test cases
# ----------------------------
if __name__ == "__main__":
    test_emails = [
        # Valid emails
        "test@example.com",
        "user.name@domain.co",
        "user_name123@sub.domain.org",
        "email+alias@gmail.com",

        # Invalid emails
        "invalid-email",              # Missing @ and domain
        "user@.com",                  # Invalid domain
        "user@domain",                # Missing TLD
        "@nouser.com",                # Missing local part
        "user@domain..com",           # Double dot in domain
    ]

    for email in test_emails:
        print(f"{email:30} -> {is_valid_email(email)}")
```

Notes:

- This regex covers **most practical use cases** while staying readable.
- It intentionally avoids full RFC 5322 complexity (which is extremely verbose and rarely needed in real systems).
- Using `re.fullmatch()` ensures **no partial matches**, which is more robust than `re.match()`.

Prompt:

- Generate an SQL query to find the top 5 customers by revenue in the last 30 days

Result:

Here's a clean and commonly used SQL query for that requirement:

```sql
SELECT

    c.customer_id,

    c.customer_name,

    SUM(o.total_amount) AS total_revenue

FROM customers c

JOIN orders o

    ON c.customer_id = o.customer_id

WHERE o.order_date >= CURRENT_DATE - INTERVAL '30 days'

GROUP BY c.customer_id, c.customer_name

ORDER BY total_revenue DESC

LIMIT 5;
```

How it works:

- **JOIN**: Links customers with their orders
- **WHERE**: Filters orders from the last 30 days
- **SUM()**: Calculates total revenue per customer
- **GROUP BY**: Aggregates revenue per customer
- **ORDER BY DESC**: Sorts highest revenue first
- **LIMIT 5**: Returns top 5 customers

Variations (depending on your DB):

- **MySQL**

```sql
WHERE o.order_date >= NOW() - INTERVAL 30 DAY
```

- **SQL Server**

```sql
WHERE o.order_date >= DATEADD(DAY, -30, GETDATE())
```

If you want, I can adapt this to your exact schema (table names, columns, or if revenue comes from line items instead of order totals).

Prompt :

- You are an expert SQL developer with strong knowledge of analytics and query optimization.
- Write an SQL query to find the top 5 customers by total revenue generated in the last 30 days.
- Requirements:
  - Assume a typical schema with:
  - customers(customer_id, customer_name)
  - orders(order_id, customer_id, order_date, total_amount)
  - Calculate revenue as the sum of total_amount from orders.
  - Only include orders from the last 30 days based on order_date.
  - Group results by customer.
- Return:
  - customer_id
  - customer_name
  - total_revenue
  - Sort results in descending order of revenue.
  - Limit the output to the top 5 customers.
  - Use clear aliases and write the query in a clean, readable format.
  - Add brief comments explaining key parts of the query.
  - If needed, assume a standard SQL dialect (e.g., PostgreSQL).

Result:

Here’s a clean, well-structured PostgreSQL query that meets all your requirements:

```sql
SELECT
    c.customer_id,
    c.customer_name,
    SUM(o.total_amount) AS total_revenue  -- Aggregate total
revenue per customer
FROM customers AS c
JOIN orders AS o
    ON c.customer_id = o.customer_id      -- Join orders with
customers
WHERE o.order_date >= CURRENT_DATE - INTERVAL '30 days'  --
Filter last 30 days
GROUP BY
    c.customer_id,
    c.customer_name                       -- Group by customer to
aggregate revenue
ORDER BY
    total_revenue DESC                    -- Sort by highest
revenue first
LIMIT 5;                                  -- Return top 5
customers
```

Key Points:

- Uses `CURRENT_DATE - INTERVAL '30 days'` for a rolling 30-day window
- `SUM(total_amount)` computes total revenue per customer
- Proper grouping + ordering ensures accurate ranking
- Clean aliases (`c`, `o`) improve readability

Prompt:

- Explain the difference between BERT and GPT models in exactly 3 bullet points.
- Requirements:
    - Use simple and clear language
    - Each bullet point should highlight one key difference

    - Focus on core concepts (e.g., architecture, training approach, use cases)
    - Keep each bullet concise (1–2 lines max)

Result:

- **Architecture:** BERT reads text in both directions (bidirectional), while GPT reads text only left-to-right (unidirectional).
- **Training approach:** BERT is trained to understand context by predicting masked words; GPT is trained to generate text by predicting the next word.
- **Use cases:** BERT is best for understanding tasks (like classification, Q&A), while GPT is best for generation tasks (like writing, chat, code).

Prompt

- Identify 3 recent (2024–2025) AI developments that could impact hotel booking systems.
- Requirements:
    - Focus only on recent and relevant advancements (2024–2025)
    - Present the answer in exactly 3 bullet points
    - Each point should briefly explain the development and its impact
    - Use simple and clear language
    - Focus on practical applications (e.g., personalization, pricing, search, automation)

Result

- **Generative AI & conversational booking (2024–2025):** AI assistants now let users search and book hotels using natural language chats, changing booking systems from form-based flows to chat-driven experiences. (Hospitality.today)
- **AI-driven personalization engines:** Modern systems analyze user behavior and preferences to offer tailored hotel recommendations and deals, significantly improving conversions and repeat bookings. (Hospitality Tech)
- **Real-time dynamic pricing with predictive analytics:** AI models adjust hotel prices automatically based on demand, competitor rates, and external factors, helping maximize revenue and occupancy. (mycloudhospitality.com)

AI performed well overall, generating a correct email validation function with minimal changes, but needed clarification for the SQL query regarding schema and date logic; it produced a clear and professional delay email with minor tone tweaks and a well-structured README that required project-specific additions; it accurately explained BERT vs GPT without changes, and identified relevant recent AI trends, though slight refinement was needed to ensure recency and practical relevance.